

A report submitted to



Affiliated To: Veer Narmad South Gujarat University, Surat

For, Partial requirement for the fulfillment of the degree of

Bachelor of Computer Applications (BCA)

BCA Sem VI

A.Y. 2025-26

Project Report On Student_performance Analysis

by

Roll No.	Seat No.	Name Of Student
403	6756	BHINGRADIYA FENIL NITINBHAI
402	6730	AVAIYA NAND RAJESHBHAI

Project Guide by: Prof. Jaimini Patel

Acknowledgement

The success and final outcome of this project required a lot of guidance and assistance from many people and we are extremely fortunate to have got this all along the completion of our project work. Whatever we have done is only due to such guidance and assistance.

We would not forget to thank Principal Dr. Aditi Bhatt, IQAC coordinator and trust representative Dr. Vaibhav Desai, In-charge of PG Courses Dr. Vimal Vaiwala , In-charge of UG-BCA and Head of the BCA Department Prof. Nainesh Gathiyawala and Project guide Prof. Jaimini Patel and all other Assistant professors of SDJ International College, Vesu, who took keen interest on our project work and guided us all along, till the completion of our project work by providing all the necessary information for developing a good system.

We are extremely grateful to her/him for providing such a nice support and guidance though she/he had busy schedule managing the college dealings.

We are thankful and fortunate enough to get support and guidance from all Teaching staffs of IT Department which helped us in successfully completing our project work. Also, we would like to extend our sincere regards to all the non-teaching staff of IT Department for their timely support.

BHINGRADIYA FENIL NITINBHAI (6756)
AVAIYA NAND RAJESHBHAI (6730)

INDEX

Sr. No	Description	Page No.
1	Task 1: Data Understanding	5
2	Task 2: Exploratory Data Analysis (EDA)	9
3	Task 3: Handling Missing Data and Outliers	13
4	Task 4: Spread of Data	18
5	Task 5: Automating EDA using Python	20
6	Task 6: Regression Analysis	22
7	Task 7: Supervised Learning – Regression Model	24
8	Task 8: Overfitting and Underfitting Analysis	26
9	Task 9: Classification Task	27
10	Task 10: Model Evaluation	28
11	Task 11: Data visualization	29
12	Task 12: Project Conclusion	33

Introduction

Student Performance Analysis

Exploratory Data Analysis (EDA) is the process of examining and summarizing a dataset to understand its main characteristics before applying statistical or machine learning models. It helps in identifying patterns, detecting missing values, finding outliers, and understanding relationships between variables.

In this project, **Python** is used for performing EDA because it provides powerful libraries such as **Pandas, NumPy, Matplotlib, and Seaborn** for data manipulation and visualization. These libraries help in cleaning the data, calculating statistical summaries, and creating graphical representations of the dataset.

The project “**Student Performance Analysis**” focuses on analyzing student's academic data, including their marks in subjects like **Math, Science, and English**, along with other factors such as **study hours and extracurricular activities**. By applying EDA techniques, the dataset is explored to understand how different factors affect students' academic performance.

Dataset Description

The dataset contains 1000 records (rows) and 12 variables (columns) representing students' demographic information, subject marks, and performance indicators. The dataset is mainly used for Exploratory Data Analysis (EDA), regression analysis, and classification tasks.

Each row represents one student, and each column represents a specific attribute related to the student's academic performance.

Summary of Dataset Variables:

Column Name	Type	Description
id	Integer	Unique student ID
age	Integer	Age of student
gender	Categorical	Gender of student
math	Numerical	Math marks
science	Numerical	Science marks
english	Numerical	English marks
sports	Categorical	Sports performance
music	Categorical	Music activity
Total Marks	Numerical	Sum of subject marks
Percentage	Numerical	Percentage score
GradeClass	Categorical	Student grade
StudyHours	Numerical	Study time

• Data Collection

Data Collection is the process of gathering data from different sources to use it for analysis and machine learning.

Sources of Data:

Data can be collected from:

- CSV files
- Databases
- APIs
- Web scraping
- Online datasets (Kaggle, UCI Repository, etc.)

File Name: student_performance.csv

Example:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# [It loads the dataset]
df=pd.read_csv('student_performance.csv')
df
```

Output:

	id	age	gender	math	science	english	sports	music	Total Marks	Percentage	GradeClass	StudyHours
0	1	20	female	91.0	63.0	43.0	No	Yes	197	65.666667	D	11
1	2	21	female	84.0	70.0	57.0	Yes	Yes	211	70.333333	C	8
2	3	19	female	71.0	61.0	63.0	No	No	195	65.000000	D	15
3	4	21	female	60.0	68.0	93.0	No	Yes	221	73.666667	C	12
4	5	21	female	80.0	45.0	75.0	No	Yes	200	66.666667	D	9
...	...	...	...	...	...	...	...	...	...	...	...	...
995	996	18	female	86.0	73.0	68.0	No	No	227	75.666667	C	14
996	997	17	female	70.0	76.0	41.0	No	Yes	187	62.333333	D	13
997	998	17	male	57.0	55.0	64.0	No	Yes	176	58.666667	E	6
998	999	20	female	68.0	77.0	89.0	Yes	Yes	234	78.000000	C	12
999	1000	19	female	59.0	70.0	73.0	No	No	202	67.333333	D	12

1000 rows × 12 columns

1. Data Understanding

Data Understanding is the step where we study and explore the dataset to know what information it contains. It helps us understand the structure, features, and quality of the data before performing analysis.

1. Purpose of Data Understanding

The purpose of data understanding is to:

- Understand the **dataset structure**
- Identify **features (columns)**
- Check **data types**
- Detect **missing values**
- Analyze **basic statistics**

This step helps us prepare the data for **data cleaning and analysis**.

2. Dataset Structure

In the dataset:

- **Rows:** represent individual records (for example, students).
- **Columns:** represent features or attributes (such as age, marks, study hours).

Example:

```
# [It displays the structure of the dataset]
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   id              1000 non-null   int64  
 1   age             1000 non-null   int64  
 2   gender          1000 non-null   object  
 3   math            950 non-null    float64 
 4   science         952 non-null    float64 
 5   english         951 non-null    float64 
 6   sports          950 non-null    object  
 7   music           1000 non-null   object  
 8   Total Marks     1000 non-null   int64  
 9   Percentage      1000 non-null   float64 
10   GradeClass     1000 non-null   object  
11   StudyHours      1000 non-null   int64  
dtypes: float64(4), int64(4), object(4)
memory usage: 93.9+ KB
```

3. Types of Data

3.1: Numerical Data

- Data represented by numbers.
- Example: age, marks, study hours.

3.2: Categorical Data

- Data represented by categories or labels.
- Example: gender, grade class.

4. Checking the Dataset in Python

df.head() is a function in pandas used to display the first few rows of a dataset.

```
# [It displays the first 5 rows]
print("First five rows of the dataset:")
df.head()
```

First five rows of the dataset:

	id	age	gender	math	science	english	sports	music	Total Marks	Percentage	GradeClass	StudyHours
0	1	20	female	91.0	63.0	43.0	No	Yes	197	65.666667	D	11
1	2	21	female	84.0	70.0	57.0	Yes	Yes	211	70.333333	C	8
2	3	19	female	71.0	61.0	63.0	No	No	195	65.000000	D	15
3	4	21	female	60.0	68.0	93.0	No	Yes	221	73.666667	C	12
4	5	21	female	80.0	45.0	75.0	No	Yes	200	66.666667	D	9

df.tail() is a function in pandas used to display the last few rows of a dataset (DataFrame).

```
# [It displays the last 5 rows]
print("Last five rows of the dataset:")
df.tail()
```

Last five rows of the dataset:

	id	age	gender	math	science	english	sports	music	Total Marks	Percentage	GradeClass	StudyHours
995	996	18	female	86.0	73.0	68.0	No	No	227	75.666667	C	14
996	997	17	female	70.0	76.0	41.0	No	Yes	187	62.333333	D	13
997	998	17	male	57.0	55.0	64.0	No	Yes	176	58.666667	E	6
998	999	20	female	68.0	77.0	89.0	Yes	Yes	234	78.000000	C	12
999	1000	19	female	59.0	70.0	73.0	No	No	202	67.333333	D	12

df.shape is an attribute in pandas used to find the size (dimensions) of a DataFrame.

```
# [It displays the shape(number of rows & columns) of the dataset]
print("Shape of the dataset:")
df.shape
```

Shape of the dataset:
(1000, 12)

df.describe() is a function in pandas used to generate statistical summary of numerical columns in a dataset.

```
# [It displays the count,mean,std,min,Q1,Q2,Q3,max]
print("[Count,Mean,Std,Mean,Q1,Q2,Q3,Max] values of Data:")
df.describe()
```

[Count,Mean,Std,Mean,Q1,Q2,Q3,Max] values of Data:

	id	age	math	science	english	Total Marks	Percentage	StudyHours
count	1000.000000	1000.000000	950.000000	952.000000	951.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	19.004000	70.783158	71.934874	67.350158	199.776000	66.592000	9.840000
std	288.819436	1.433192	16.312973	14.924907	14.083002	38.577818	12.859273	3.15854
min	1.000000	17.000000	15.000000	20.000000	18.000000	52.000000	17.333333	5.000000
25%	250.750000	18.000000	61.000000	63.000000	58.000000	183.000000	61.000000	7.000000
50%	500.500000	19.000000	71.000000	72.000000	68.000000	205.000000	68.333333	10.000000
75%	750.250000	20.000000	81.000000	81.000000	77.000000	224.250000	74.750000	13.000000
max	1000.000000	21.000000	118.000000	127.000000	109.000000	295.000000	98.333333	15.000000

df.columns is an attribute in pandas used to display the names of all columns in a DataFrame.

```
# [It displays the column names]
print("Column names:")
list(df.columns)
```

```
Column names:
['id',
 'age',
 'gender',
 'math',
 'science',
 'english',
 'sports',
 'music',
 'Total Marks',
 'Percentage',
 'GradeClass',
 'StudyHours']
```

df.select_dtypes() is a function in pandas used to select columns based on their data types.

Quantitative data is data that is numerical and can be measured or counted.

```
# [It displays the quantitative data columns of the dataset]
print("Quantitative Data Columns:")
quantitative=df.select_dtypes(include=['int64','float64']).columns
print(quantitative)
```

```
Quantitative Data Columns:
Index(['id', 'age', 'math', 'science', 'english', 'Total Marks', 'Percentage',
       'StudyHours'],
      dtype='object')
```

Qualitative data is data that describes qualities or categories and is not numerical.

```
# [It displays the qualitative data columns of the dataset]
print("Qualitative Data Columns:")
qualitative=df.select_dtypes(include=['object']).columns
print(qualitative)
```

```
Qualitative Data Columns:
Index(['gender', 'sports', 'music', 'GradeClass'], dtype='object')
```

2. Exploratory Data Analysis (EDA)

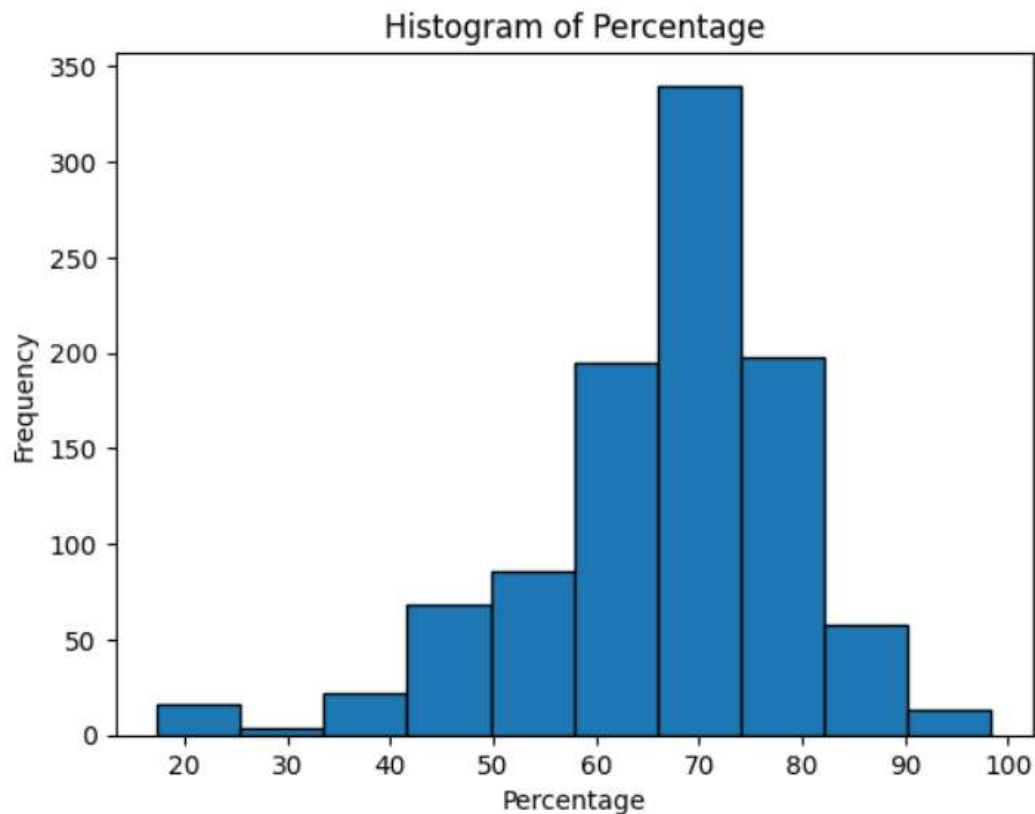
Exploratory Data Analysis (EDA) is the process of analyzing and visualizing the dataset to discover patterns, relationships, and important information.

1. Univariate Analysis:

Univariate Analysis is the analysis of one variable at a time in a dataset.

```
# [Histogram]
print("1. Univariate Analysis of Percentage")
plt.figure()
plt.hist(df["Percentage"], bins=10, edgecolor='black')
plt.xlabel("Percentage")
plt.ylabel("Frequency")
plt.title("Histogram of Percentage")
plt.show()
```

1. Univariate Analysis of Percentage

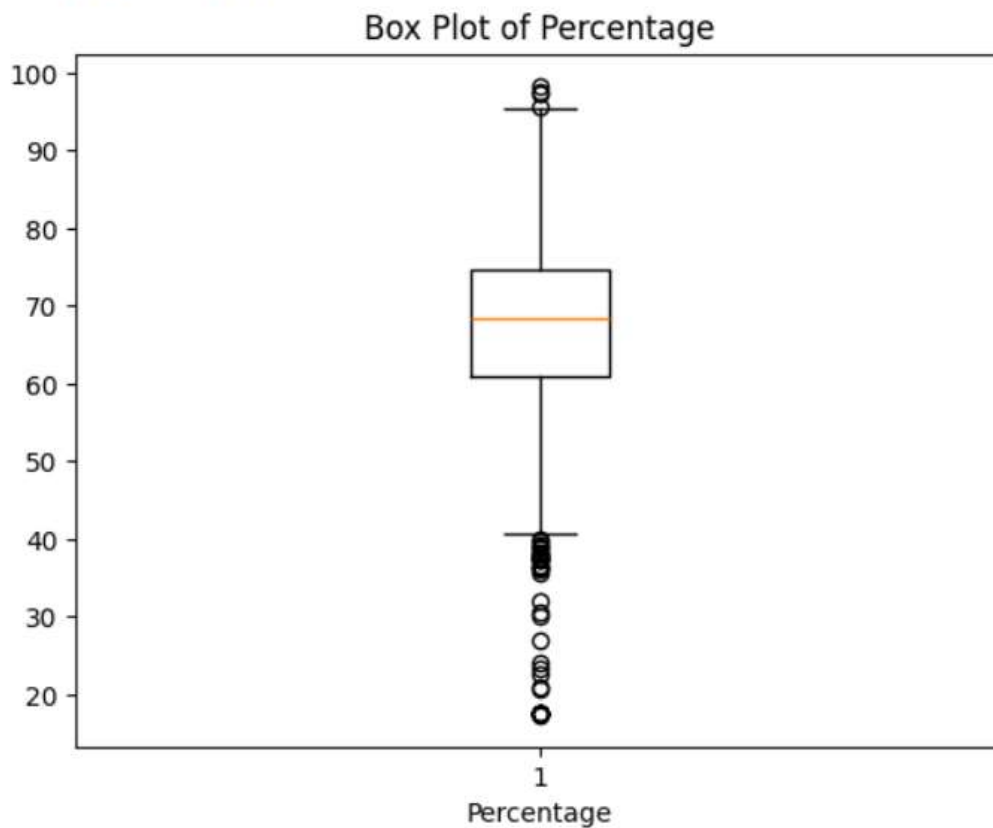


1. Univariate Analysis:

Univariate Analysis is the analysis of one variable at a time in a dataset.

```
# [Box Plot]
print("2. Univariate Analysis of Percentage")
plt.figure()
plt.boxplot(df["Percentage"])
plt.xlabel("Percentage")
plt.title("Box Plot of Percentage")
plt.show()
```

2. Univariate Analysis of Percentage

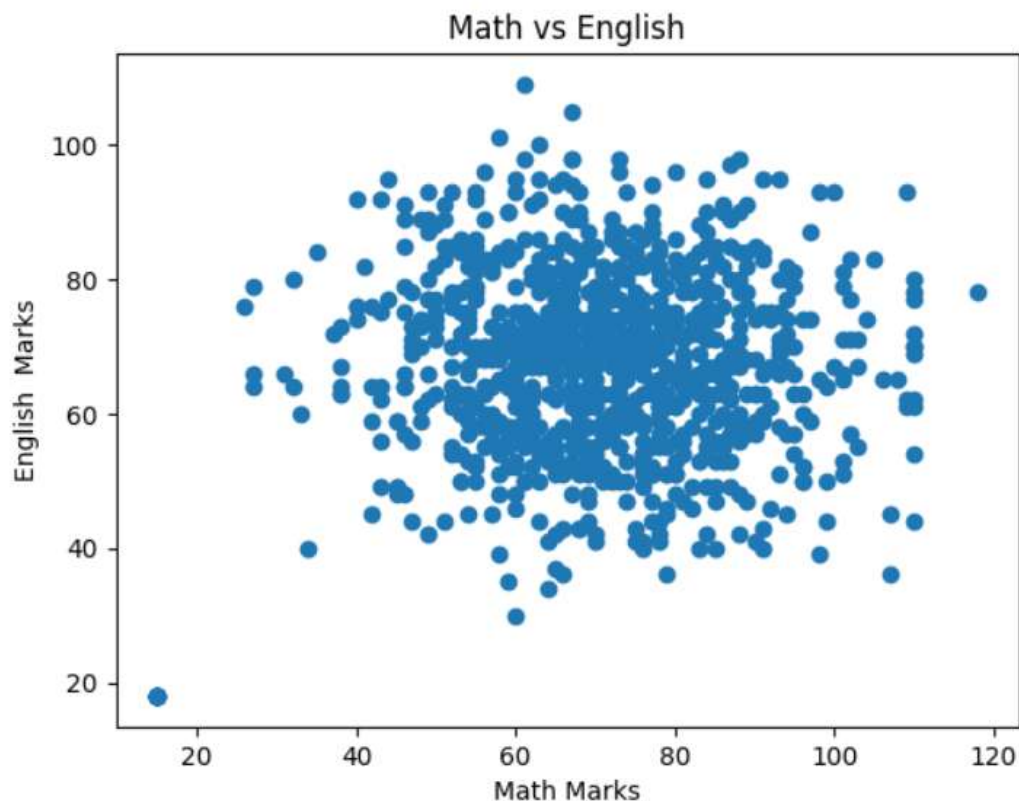


2. Bivariate Analysis:

Bivariate Analysis is the analysis of two variables at the same time to understand the relationship between them.

```
# [Scatter Plot]
print("1. Bivariate Analysis of Math Vs English:")
plt.scatter(df["math"], df["english"])
plt.xlabel("Math Marks")
plt.ylabel("English Marks")
plt.title("Math vs English")
plt.show()
```

1. Bivariate Analysis of Math Vs English:



2. Bivariate Analysis:

Bivariate Analysis is the analysis of two variables at the same time to understand the relationship between them.

```
# [Correlation]
print("2. Bivariate Analysis of Math Vs Science")
num_cols = ["math", "science"]
df[num_cols].corr()
```

2. Bivariate Analysis of Math Vs Science

	math	science
math	1.000000	0.189352
science	0.189352	1.000000

3. Multivariate Analysis:

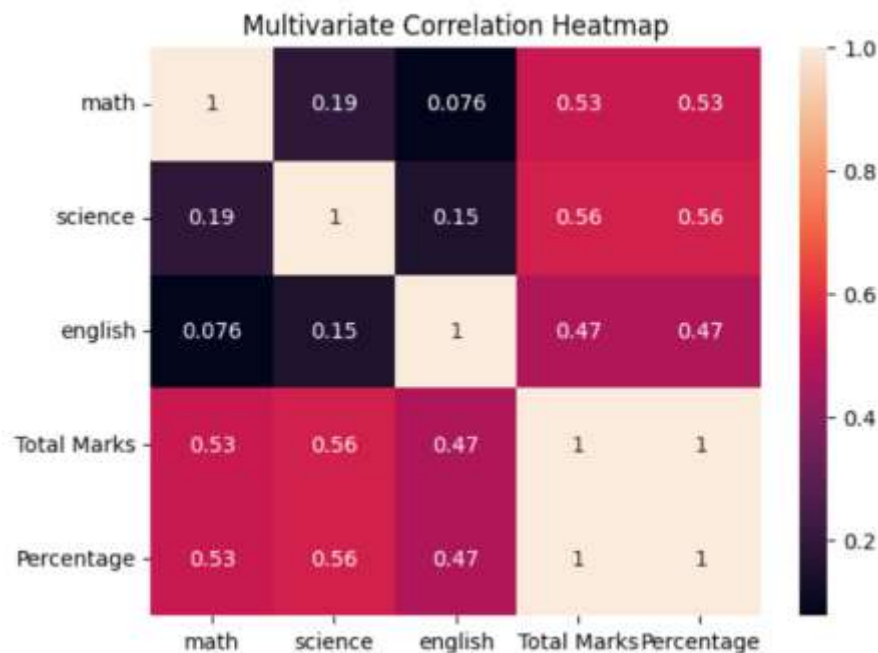
Multivariate Analysis is a statistical technique used to analyze more than two variables at the same time to understand relationships, patterns, or effects between them.

```
print("1. Multivariate Analysis of more than two variables:")
# [Select important numerical columns]
selected_cols = ["math", "science", "english",
                 "Total Marks", "Percentage"]

# [Correlation matrix]
corr_matrix = df[selected_cols].corr()

# [Heatmap]
plt.figure()
sns.heatmap(corr_matrix, annot=True)
plt.title("Multivariate Correlation Heatmap")
plt.show()
```

1. Multivariate Analysis of more than two variables:



3. Handling Missing Data and Outliers

3.1 Handling Missing Data:

Handling Missing Data means the process of detecting and managing missing values (NaN or empty values) in a dataset so that the data can be analyzed correctly.

```
# [identifying missing values]  
df.isnull().sum()
```

```
id          0  
age         0  
gender      0  
math       50  
science    48  
english    49  
sports     50  
music       0  
Total Marks 0  
Percentage  0  
GradeClass  0  
StudyHours  0  
dtype: int64
```

In mathematics and data analysis, NaN (Not a Number) represents missing values. When calculating the mean (average) of a column, NaN values are ignored.

```
# [It displays the mean of the NA values in the specific column]  
  
print("Mean of the column Math:")  
df["math"].fillna(df["math"].mean())  
#df["science"].fillna(df["science"].mean())  
#df["english"].fillna(df["english"].mean())
```

Mean of the column Math:

```
0      91.0  
1      84.0  
2      71.0  
3      60.0  
4      80.0  
...  
995    86.0  
996    70.0  
997    57.0  
998    68.0  
999    59.0  
Name: math, Length: 1000, dtype: float64
```

Median is the middle value of a dataset after arranging the values in ascending order. When calculating the median, NaN (missing values) are ignored.

```
# [It displays the median of the NA values in the specific column]

#df["math"].fillna(df["math"].median())
print("Median of the column Science:")
df["science"].fillna(df["science"].median())
#df["english"].fillna(df["english"].median())
```

Median of the column Science:

```
0      63.0
1      70.0
2      61.0
3      68.0
4      45.0
...
995    73.0
996    76.0
997    55.0
998    77.0
999    70.0
```

Name: science, Length: 1000, dtype: float64

Mode is the value that appears most frequently in a dataset. When calculating the mode, NaN (missing values) are ignored.

```
# [It displays the mode of the NA values in the specific column]

#df["math"].fillna(df["math"].mode()[0])
#df["science"].fillna(df["science"].mode()[0])
print("Mode of the column english:")
df["english"].fillna(df["english"].mode()[0])
```

Mode of the column english:

```
0      43.0
1      57.0
2      63.0
3      93.0
4      75.0
...
995    68.0
996    41.0
997    64.0
998    89.0
999    73.0
```

Name: english, Length: 1000, dtype: float64

fillna() is used to replace missing values (NaN) with another value such as "NA" in a column.

```
# [It fills the null values using 'NA' in sports column]  
df['sports'].fillna("NA")
```

```
0      No  
1      Yes  
2      No  
3      No  
4      No  
...  
995    No  
996    No  
997    No  
998    Yes  
999    No  
Name: sports, Length: 1000, dtype: object
```

3.2 Outliers Detection

Outlier Detection is the process of identifying data points that are very different from the rest of the data in a dataset.

```
# [Find the Outliers from the specific columns]
# [Select subject columns]
subjects = ["math", "science", "english"]

for col in subjects:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"\nOutliers in {col}:")
    print(outliers[[col]])
```

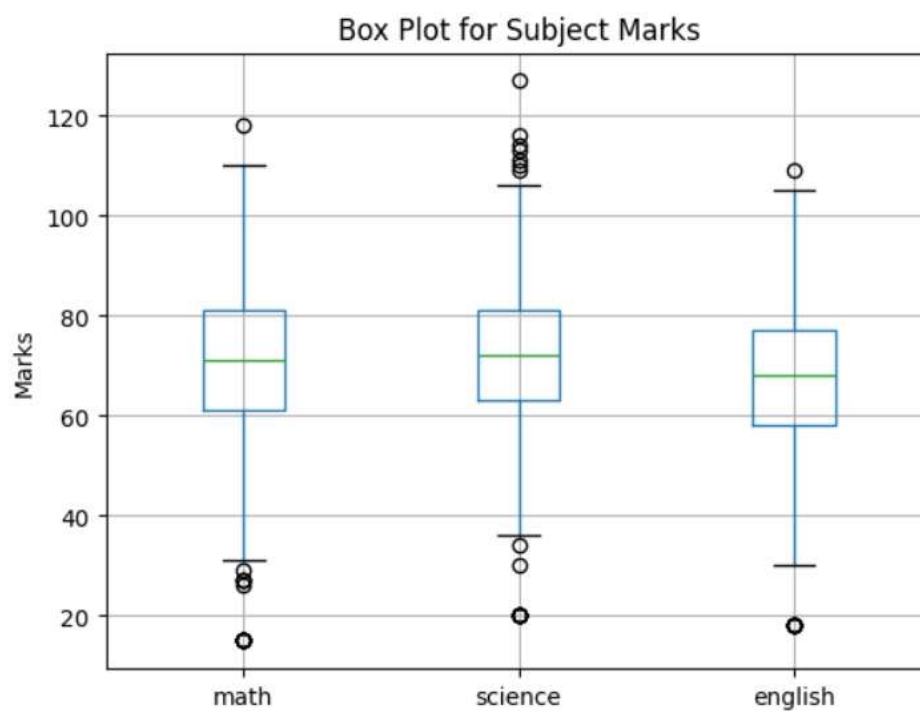
Output:

Outliers in math:	Outliers in science:	Outliers in english:
math	science	english
10 15.0	10 20.0	10 18.0
61 27.0	35 111.0	102 18.0
101 27.0	102 20.0	309 18.0
102 15.0	305 116.0	383 18.0
309 15.0	309 20.0	445 18.0
383 15.0	383 20.0	469 18.0
445 15.0	445 20.0	707 18.0
469 15.0	469 20.0	929 18.0
539 27.0	506 113.0	933 18.0
591 26.0	521 114.0	976 18.0
615 118.0	574 109.0	982 109.0
707 15.0	707 20.0	
750 29.0	801 110.0	
929 15.0	884 34.0	
933 15.0	895 127.0	
976 15.0	929 20.0	
	933 20.0	
	947 30.0	
	976 20.0	

3.2 Outliers detection using Boxplot

A box plot is a graphical method used to identify outliers in a dataset using quartiles and the interquartile range (IQR).

```
# [Outlier detection using Boxplot]
plt.figure()
df[subjects].boxplot()
plt.title("Box Plot for Subject Marks")
plt.ylabel("Marks")
plt.show()
```

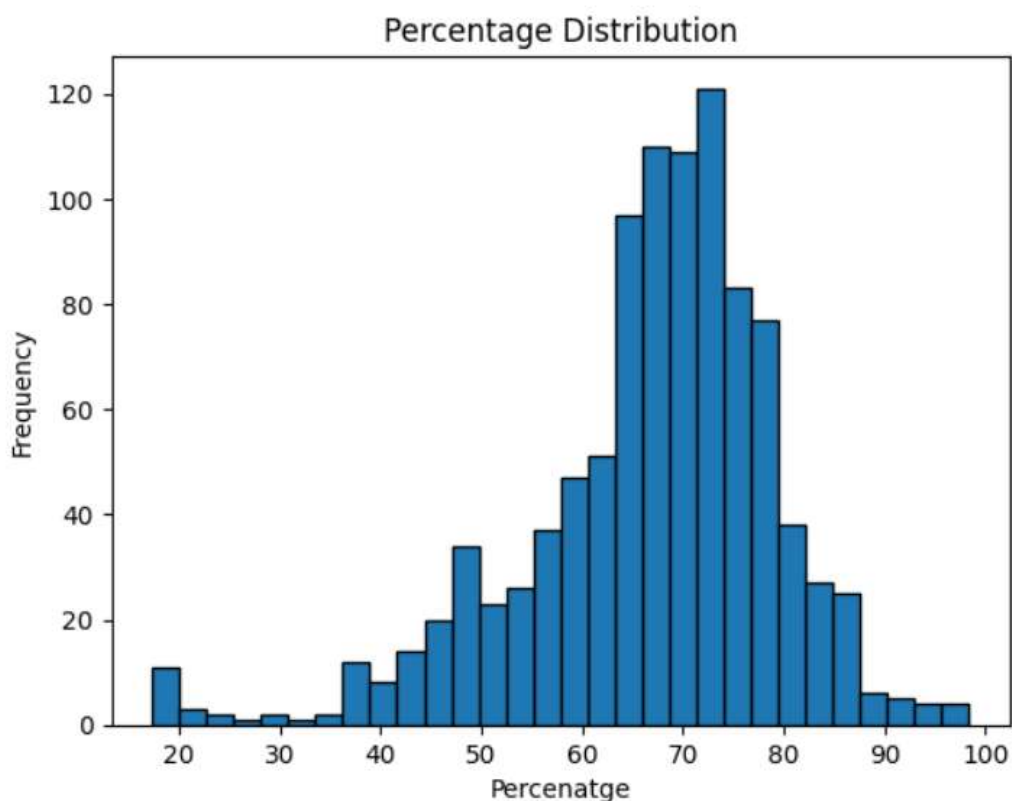


4. Spread of Data

4.1 Ccheck Data Distribution

Spread of data refers to how much the values in a dataset vary or are distributed from the center (mean or median).

```
# [It displays the negative skewed distribution of the Percentage]
plt.figure()
plt.hist(df['Percentage'],bins=30, edgecolor='black')
plt.title('Percentage Distribution')
plt.xlabel('Perceinatge')
plt.ylabel('Frequency')
plt.show()
```



Most students have percentages between 60% and 80%, indicating that many students perform in the average to good range. The distribution is negatively skewed (left-skewed), meaning most values are on the higher side while fewer students have lower percentages.

4.2 Calculations:

- **Mean:**

```
# [It displays the average value of the Percentage values]
mean_value=df['Percentage'].mean()
print("Mean of the Percentage column:",mean_value)
```

Mean of the Percentage column: 66.59200000002001

- **Median:**

```
# [It displays the middle value of the Percentage values]
median_value=df['Percentage'].median()
print("Median of the Percentage column:",median_value)
```

Median of the Percentage column: 68.33333333

- **Standard Deviation:**

```
# [It displays how spread the data is from the mean]
std_value=df['Percentage'].std()
print("Standard Deviation of the Percentage column:",std_value)
```

Standard Deviation of the Percentage column: 12.859272797958495

- **Skewness:**

```
# [It indicates the direction of the data skew(left or right)]
skew_value=df['Percentage'].skew()
print("Skewness is:",skew_value)
```

Skewness is: -1.0585706328809699

- **Kurtosis:**

```
# [It indicates the presence of the outliers]
kurt_value=df['Percentage'].kurt()
print("Kurtosis is:",kurt_value)
```

Kurtosis is: 2.066163642103207

5. Automating EDA using Python

Automating Exploratory Data Analysis (EDA) using Python refers to the process of automatically analyzing and summarizing a dataset using Python libraries and tools instead of performing each step manually. It helps data analysts quickly understand the structure, patterns, relationships, distributions, and potential issues in the data.

```
import pandas as pd
from ydata_profiling import ProfileReport

data = pd.read_csv("student_performance.csv")

profile = ProfileReport(data)

profile.to_file("EDA_Report.html")
```

[Upgrade to ydata-sdk](#)

Improve your data and profiling with ydata-sdk, featuring data quality scoring, redundancy detection, outlier identification, text validation, and synthetic data generation.

Summarize dataset: 100%  71/71 [00:09<00:00, 4.67it/s, Completed]

100%  12/12 [00:00<?, 31it/s]

Generate report structure: 100%  1/1 [00:05<00:00, 5.44s/it]

Render HTML: 100%  1/1 [00:01<00:00, 1.49s/it]

Export report to file: 100%  1/1 [00:00<00:00, 27.70it/s]

- **Reusable Python Function:**

A reusable Python function is a function that can be used multiple times in a program without rewriting the same code again.

It is created using the def keyword and helps make the code simpler, organized, and efficient.

```
# [Write the reusable function for EDA using "def" keyword]
def basic_info(df):
    print("Shape:", df.shape)
    print("\nColumns:\n", df.columns)
    print("\nData Types:\n", df.dtypes)
    print("\nMissing Values:\n", df.isnull().sum())

basic_info(df)

Shape: (1000, 12)

Columns:
Index(['id', 'age', 'gender', 'math', 'science', 'english', 'sports', 'music',
       'Total Marks', 'Percentage', 'GradeClass', 'StudyHours'],
      dtype='object')

Data Types:
id          int64
age         int64
gender      object
math        float64
science     float64
english     float64
sports      object
music       object
Total Marks int64
Percentage  float64
GradeClass  object
StudyHours  int64
dtype: object

Missing Values:
id          0
age         0
gender      0
math        0
science     48
english     49
sports      50
music       0
Total Marks 0
Percentage  0
GradeClass  0
StudyHours  0
dtype: int64
```

6. Regression Analysis

Regression analysis is a statistical method used to study the relationship between a dependent variable and one or more independent variables. It helps to predict the value of the dependent variable based on the independent variables.

```
# [Regression Analysis (Identify dependent variable & independent variable)]
print("Dependent Variable[Y] : Percentage")
print("Independent Variable[X] : math,science,english")
```

```
Dependent Variable[Y] : Percentage
Independent Variable[X] : math,science,english
```

6.1 Linear Regression

Linear Regression is a machine learning technique used to predict the value of a dependent variable based on the relationship with an independent variable using a straight-line equation.

```
print("Linear Regression:")
print("-----")
from sklearn.linear_model import LinearRegression

# [Clean column names]
df.columns = df.columns.str.strip()

# [Convert to numeric]
df['math'] = pd.to_numeric(df['math'], errors='coerce')
df['Percentage'] = pd.to_numeric(df['Percentage'], errors='coerce')

# Replace missing values with mean
df['math'] = df['math'].fillna(df['math'].mean())
df['Percentage'] = df['Percentage'].fillna(df['Percentage'].mean())

# Define X and y
X = df[['math']] # Independent variable
y = df['Percentage'] # Dependent variable

# Create and train model
model = LinearRegression()
model.fit(X, y)

# Print regression equation values
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)

# Predict for math = 90
prediction = model.predict([[90]])[0]
print("Predicted Percentage:", prediction)

Linear Regression:
-----
Slope: 0.38622681474822806
Intercept: 39.253646388514895
Predicted Percentage: 74.01405971585541
```

6.2 Analyze Covariance & Correlation:

- **Covariance:**

This code calculates the **covariance between the math scores and percentage**, which measures how the two variables change together in the dataset.

```
# [Covariance between math and Percentage]
cov_value = df['math'].cov(df['Percentage'])

print("Covariance:", cov_value)
```

Covariance: 97.63586955100936

- **Correlation:**

This code calculates the **correlation between math scores and percentage**, which measures the **strength and direction of the relationship between the two variables**.

```
# [Correlation between math and Percentage]
corr_value = df['math'].corr(df['Percentage'])

print("Correlation:", corr_value)
```

Correlation: 0.47753972047591525

7. Supervised Learning – Regression Model

Supervised Learning – Regression Analysis is a machine learning technique where a model learns from labeled data to predict continuous numerical values. The algorithm is trained using input features (independent variables) and known outputs (dependent variables) to learn the relationship between them.

```
print("Supervised Learning - Regression Model:")
print("-----")
# Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

# Step 2: Load Dataset
df = pd.read_csv("student_performance.csv")

# Step 3: Select Features (X) and Target (y)
X = df[["StudyHours"]] # Independent Variable
y = df["Percentage"]   # Dependent Variable

# Step 4: Split Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 5: Create Model
model = LinearRegression()

# Step 6: Train Model
model.fit(X_train, y_train)

# Step 7: Make Predictions
y_pred = model.predict(X_test)

# Step 8: Evaluate Model
print("R2 Score:", r2_score(y_test, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))

# Step 9: Predict for New Student
study_hours = [[8]]
predicted_marks = model.predict(study_hours)
print("Predicted Percentage for 8 Study Hours:", predicted_marks[0])

# Step 10: Plot Graph
plt.scatter(X, y)
plt.plot(X, model.predict(X), linestyle='--')
plt.xlabel("Study Hours")
plt.ylabel("Percentage")
plt.title("Study Hours vs Percentage")
plt.show()
```

Output:

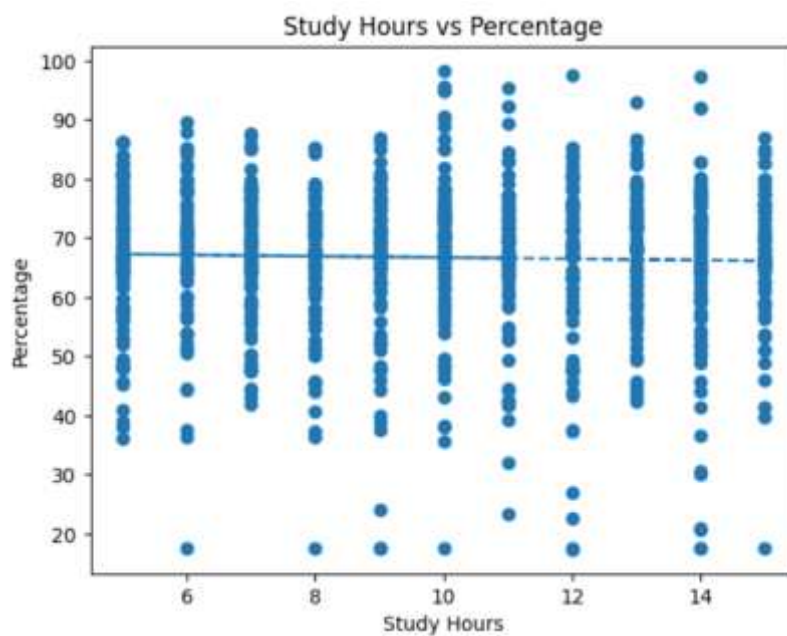
Supervised Learning - Regression Model:

R2 Score: -0.004357032157248142

Mean Absolute Error: 9.490245704623248

Predicted Percentage for 8 Study Hours: 66.95497034809277

Graph:



8. Overfitting and Underfitting Analysis

In machine learning models (like the regression model in your code), we evaluate whether the model learns the data properly or not. Two common problems are Overfitting and Underfitting.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor

# sample dataset
X = [[1],[2],[3],[4],[5],[6],[7],[8]]
y = [10,20,30,40,50,60,70,80]

# split dataset
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3)

# Underfitting model
model1 = LinearRegression()
model1.fit(X_train, y_train)

# Overfitting model
model2 = DecisionTreeRegressor()
model2.fit(X_train, y_train)

print("Linear Regression Training Score:", model1.score(X_train,y_train))
print("Linear Regression Testing Score:", model1.score(X_test,y_test))

print("Decision Tree Training Score:", model2.score(X_train,y_train))
print("Decision Tree Testing Score:", model2.score(X_test,y_test))
```

Linear Regression Training Score: 1.0
Linear Regression Testing Score: 1.0
Decision Tree Training Score: 1.0
Decision Tree Testing Score: 0.8392857142857143

9. Classification Task

A classification task is a type of supervised machine learning problem where the model predicts categorical or class labels instead of numerical values.

```
print("Classification Task:")
print("-----")
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import pandas as pd

# Example dataset
data = {
    "Hours": [2,4,6,8,10],
    "Pass": [0,0,1,1,1]
}

df = pd.DataFrame(data)

X = df[["Hours"]]
y = df["Pass"]

# Split dataset
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

# Train model
model = LogisticRegression()
model.fit(X_train,y_train)

# Prediction
pred = model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,pred))
```

```
Classification Task:
-----
Accuracy: 0.0
```

10. Model Evaluation

Model Evaluation is the process of measuring how well a trained machine learning model performs on unseen test data using evaluation metrics.

```
print("Model Evaluation:")
print("-----")
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset
df = pd.read_csv("student_performance.csv")

# Features and target
X = df[['StudyHours']]
y = df['Total Marks']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation metrics
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("MAE:", mae)
print("R2 Score:", r2)
```

```
Model Evaluation:
-----
MSE: 1557.6875911843576
MAE: 29.294140795593634
R2 Score: -0.005239014345103099
```

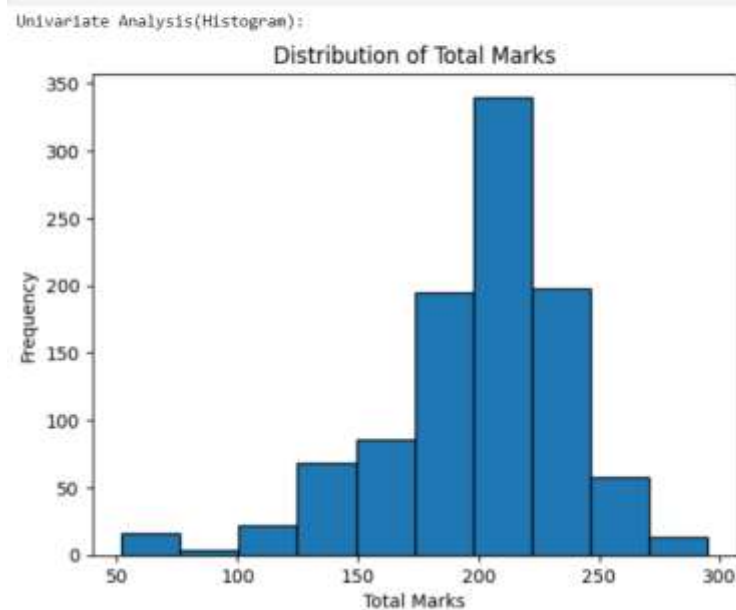
11. Data Visualization

Data visualization is the graphical representation of data using visual elements like charts, graphs, and plots to make information easier to understand and analyze.

- **Histogram:**

```
print("Univariate Analysis(Histogram): ")
import matplotlib.pyplot as plt

plt.hist(df['Total Marks'], edgecolor='black')
plt.title("Distribution of Total Marks")
plt.xlabel("Total Marks")
plt.ylabel("Frequency")
plt.show()
```

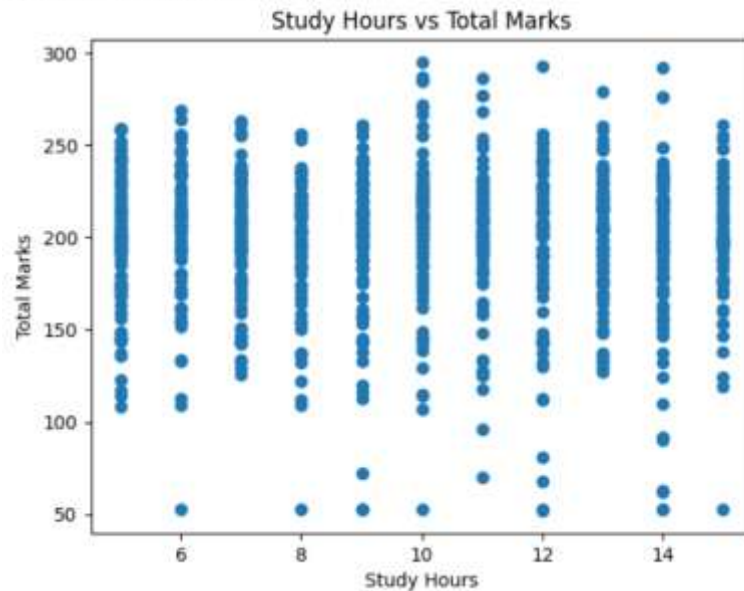


A histogram is a graphical representation used to show the distribution of a single variable. In this chart, it displays how frequently different Total Marks values occur in the dataset, helping to understand the spread and pattern of marks.

- **Scatter Plot:**

```
print("Bivariate Analysis(Scatter Plot):")  
plt.scatter(df['StudyHours'], df['Total Marks'])  
plt.title("Study Hours vs Total Marks")  
plt.xlabel("Study Hours")  
plt.ylabel("Total Marks")  
plt.show()
```

Bivariate Analysis(Scatter Plot):



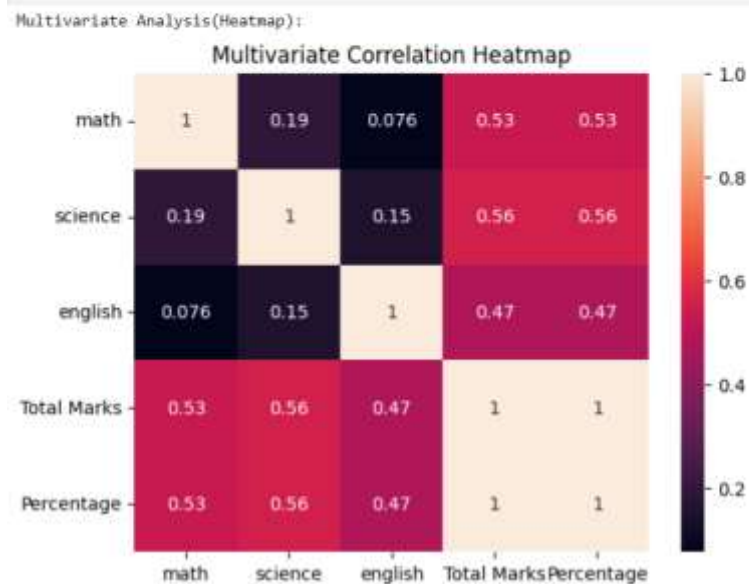
A scatter plot is used to show the relationship between two variables. In this graph, it displays the relationship between Study Hours and Total Marks, helping to understand how study time affects students' marks.

- **Heatmap:**

```
print("Multivariate Analysis(Heatmap):")
# [Select important numerical columns]
selected_cols = ["math", "science", "english",
                 "Total Marks", "Percentage"]

# [Correlation matrix]
corr_matrix = df[selected_cols].corr()

# [Heatmap]
plt.figure()
sns.heatmap(corr_matrix, annot=True)
plt.title("Multivariate Correlation Heatmap")
plt.show()
```

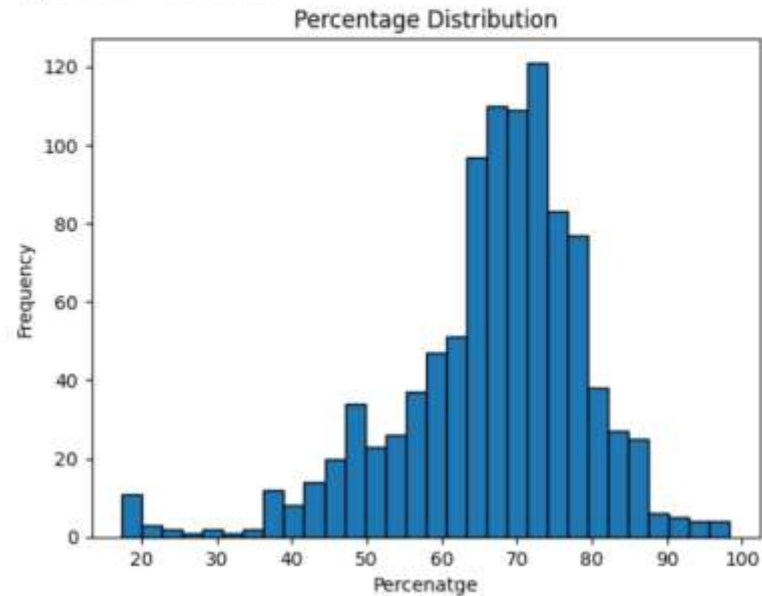


A heatmap is a visualization used to show the correlation between multiple variables using colors. In this chart, it displays the relationship between math, science, english, total marks, and percentage, helping to identify how strongly these variables are related.

- **Negative Skewed Distribution:**

```
print("Negative Skewed Distribution:")
plt.figure()
plt.hist(df['Percentage'], bins=30, edgecolor='black')
plt.title('Percentage Distribution')
plt.xlabel('Perceatage')
plt.ylabel('Frequency')
plt.show()
```

Negative Skewed Distribution:



A negative skewed distribution is a distribution where the tail extends toward the left side and most of the values are concentrated on the right side. In this histogram, most students have higher percentages, while only a few have lower percentages.

Project Conclusion: Student Performance Analysis

This project analyzed students' academic data using Exploratory Data Analysis (EDA) in Python. Various techniques such as univariate, bivariate, and multivariate analysis were used to understand the patterns and relationships in the dataset.

The histogram analysis showed the distribution of total marks and percentages, helping to understand how students' scores are spread. The scatter plot revealed the relationship between study hours and total marks, indicating that students who study more tend to achieve higher marks. The heatmap correlation analysis showed a positive relationship between subjects like math, science, and english with total marks and percentage, meaning performance in these subjects contributes significantly to overall results.